

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

Asignatura: Aplicaciones Web — Período académico 2026–2027

Sistema de Gestión Bibliotecaria Web (SGB-SaaS)

Tercera Entrega — Consolidación, Reproducibilidad Automática
y Evidencia Empírica Cuantitativa

Java 21 / Spring Boot 4

Angular 17

PostgreSQL 16

Redis 7

OWASP

Integrantes:

Cajas Ibarra Irvin Marcelo	C.I.: 1251039606
Loor Medranda Marlon Taylor	C.I.: 0928087469
Panama Murillo Moises Antonio	C.I.: 1251334544

Docente:

Ing. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Fecha de cierre:

24 de julio de 2026

Repositorio:

<https://github.com/mloorm14/sgb-saas> — Rama: main

Tag de esta entrega: v0.9.0-rc (release candidate)

Commit taguado (completo):

c6b372c9cdadee878ae5be004a00835bec4e3245a

Commit (corto): c6b372c

DOI:

<https://doi.org/10.5281/zenodo.21712467>

Índice

1. Resumen ejecutivo	2
2. Estado del sistema	3
2.1. Pila tecnológica	3
2.2. Módulos funcionales	3
2.3. Inventario de endpoints y roles requeridos	4
2.4. Decisiones arquitectónicas documentadas	4
3. Arquitectura del sistema (Modelo C4)	5
3.1. Nivel 1 – Contexto del sistema	5
3.2. Nivel 2 – Contenedores	5
3.3. Nivel 3 – Componentes	6
3.4. Atributos de calidad (ISO/IEC 25010)	6
4. Matriz de trazabilidad	7
4.1. Resumen agregado	7
4.2. Cruce prioridad × estado	7
4.3. Trazabilidad de los dos fixes de seguridad de esta entrega	8
5. Protocolo experimental	8
5.1. Determinismo y reproducibilidad	8
5.2. Estado de preparación por sub-bloque del Bloque C	9
5.3. Por qué C.1, C.3 y C.5 no se ejecutaron en esta entrega	9
6. Resultados por bloque	9
6.1. Seguridad (OWASP Top 10) – Bloque C.2	9
6.2. Cobertura de código (JaCoCo) – Bloque C.4	12
6.3. Rendimiento – Bloque C.1 (k6)	14
6.4. Usabilidad – Bloque C.3 (SUS)	14
6.5. Accesibilidad – Bloque C.5 (Lighthouse)	15
7. Amenazas a la validez	15
8. Contribuciones y declaración ética	16
8.1. Contribuciones (taxonomía CRediT)	16
8.2. Declaración ética	17
9. Referencias	17

1 Resumen ejecutivo

SGB-SaaS es un sistema web de gestión bibliotecaria construido como Proyecto Fin de Curso (PFC) de la asignatura Aplicaciones Web, con una arquitectura de tres capas: frontend SPA en Angular 17, backend API REST en Spring Boot 4.0.6 (Java 21), persistencia en PostgreSQL 16 y una capa de caché/blacklist de tokens en Redis 7. El sistema modela el ciclo completo de una biblioteca institucional: catálogo de libros, préstamos, reservaciones, multas y administración de usuarios bajo control de acceso basado en roles (RBAC).

Esta Tercera Entrega consolida el trabajo iniciado en la Entrega 1B (tag `v0.1.0-entrega-1b`) sobre cinco frentes concretos:

1. **RBAC normalizado.** El antiguo campo de texto libre `rol` se reemplazó por un modelo relacional completo (`roles`, `usuario_rols`, `permisos`, `rol_permisos`) con cuatro roles (`LECTOR`, `BIBLIOTECARIO`, `GERENTE`, `ADMIN`) aplicados consistentemente vía `@PreAuthorize` en cada endpoint (ADR-010).
2. **Estrategia híbrida de acceso a datos.** Las operaciones CRUD elementales usan Spring Data JPA; las operaciones con lógica de negocio transaccional multi-tabla (registrar devolución, pagar multa, anular multa, crear préstamo) se implementan como procedimientos almacenados de PostgreSQL, documentados en `docs/basedatos/CATALOGO-SP.md` y formalizados en ADR-013.
3. **Módulo funcional de Préstamos, Reservaciones y Multas.** Construido sobre el modelo de datos normalizado, con 5 procedimientos almacenados propios y su correspondiente capa de controladores REST (`PrestamoController`, `ReservacionController`, `MultaController`).
4. **Seguridad reforzada mediante auditoría activa.** No solo se documentó la postura de seguridad frente a OWASP Top 10 – se ejecutaron 6 auditorías en vivo contra el stack real, y dos hallazgos verdaderos que esas auditorías destaparon (ausencia de rate limiting en login, ausencia total de logging de eventos de autenticación) se corrigieron dentro de esta misma entrega, con evidencia de antes/después.
5. **Reproducibilidad automática.** Scripts dedicados (`scripts/mediciones-header.sh`, `scripts/validate-traceability.sh`, `scripts/build-init-sql.sh`) garantizan que cualquier evidencia nueva capture versiones exactas de herramientas y que la matriz de trazabilidad se valide automáticamente en CI.

Estado real, sin maquillar

Con la misma honestidad que exige la guía de esta entrega (Bloque B.2: “la edición manual de un archivo crudo de mediciones invalida la evidencia”), este informe distingue explícitamente lo que se completó de lo que no:

Completo, con evidencia real verificable: seguridad (auditoría OWASP de 6 controles + 2 correcciones en vivo), cobertura de código (JaCoCo real, 75,85 % de líneas

sobre el dominio/servicios), matriz de trazabilidad (30 requisitos, validación automática en CI), 13 Architecture Decision Records ¹, licencia MIT, colección Postman con 39 requests verificados en vivo.

Con evidencia parcial o diferida, declarado explícitamente: TLS en tránsito (OWASP A02, entorno de desarrollo sobre HTTP plano por diseño – exigido recién en Entrega Final), *Content-Security-Policy* (OWASP A05), y tres de los cinco sub-bloques de evidencia empírica del Bloque C – rendimiento (k6), usabilidad (SUS) y accesibilidad (Lighthouse) – que quedan **no ejecutados** en esta entrega por restricciones reales de tiempo del equipo, con toda su configuración y protocolo ya preparados para ejecutarse antes de la Entrega Final (ver [Sección 5](#) y [Sección 6](#)).

2 Estado del sistema

2.1 Pila tecnológica

Capa	Tecnología	Rol
Frontend	Angular 17 (TypeScript)	SPA, formularios reactivos, sesión en memoria
Backend	Spring Boot 4.0.6 / Java 21	API REST, JWT+RBAC, lógica de negocio
Base de datos	PostgreSQL 16	26 tablas, 7 procedimientos/funciones SQL
Caché/Sesión	Redis 7	Blacklist de JWT revocados, caché del catálogo
Orquestación	Docker Compose	4 contenedores, imágenes pinadas por digest

La elección de cada componente de esta pila, con alternativas consideradas y descartadas, está formalizada en ADR-001 (pila principal), ADR-011 (PostgreSQL sobre MySQL/MongoDB) y ADR-012 (Docker Compose sobre Kubernetes) – este informe no duplica ese detalle, remite a `docs/adr/` para el razonamiento completo de cada decisión.

2.2 Módulos funcionales

- **Auth** (`/api/auth/*`): registro, login, logout, refresco de `accessToken` vía cookie `HttpOnly`. Rate limiting de intentos fallidos (Redis, 5 intentos / 900 s) y bitácora de auditoría (`bitacora_auditoria`) para `LOGIN_OK`/`LOGIN_FAIL`/`LOGOUT`.
- **Libros** (`/api/v1/libros`): CRUD completo vía Spring Data JPA, caché Redis de la operación de listado con TTL configurable externamente (`CACHE_LIBROS_TTL_SECONDS`).
- **Préstamos** (`/api/v1/prestamos`): creación y devolución vía procedimientos almacenados transaccionales (`sp_crear_prestamo`, `sp_registrar_devolucion`), listados por usuario, reporte de libros más prestados.

¹Nota de precisión: el repositorio contiene actualmente 10 archivos de ADR (`docs/adr/`), no 13 – ver [Sección 2](#) para el detalle exacto. Se corrige aquí la cifra en vez de repetir un número que no coincide con el estado real del repositorio, siguiendo el mismo criterio de honestidad que exige el resto de esta entrega.

- **Reservaciones** (/api/v1/reservaciones): creación y listado por usuario; expiración automática de reservaciones vencidas vía `ReservacionScheduler` (job programado).
- **Multas** (/api/v1/multas): listado, pago y anulación (`sp_pagar_multa`, `sp_anular_multa`), esta última restringida a **GERENTE**/ADMIN exclusivamente.

2.3 Inventario de endpoints y roles requeridos

Inventario completo de los 19 endpoints REST del backend, verificado línea por línea contra los 5 `@RestController` reales (no inferido) – misma fuente usada para construir `docs/postman/coleccion.json`.

Método y ruta	Roles
POST /api/auth/registro	público
POST /api/auth/login	público
POST /api/auth/refresh	público (cookie)
POST /api/auth/logout	autenticado
GET /api/v1/libros	LECTOR+
GET /api/v1/libros/{id}	LECTOR+
POST /api/v1/libros	BIBLIOTECARIO+
PUT /api/v1/libros/{id}	BIBLIOTECARIO+
DELETE /api/v1/libros/{id}	BIBLIOTECARIO+
POST /api/v1/prestamos	BIBLIOTECARIO/GERENTE
POST /api/v1/prestamos/{id}/devolucion	BIBLIOTECARIO/GERENTE
GET /api/v1/prestamos/usuario/{id}	LECTOR/BIBLIOTECARIO/GERENTE
GET /api/v1/prestamos/usuario/{id}/activos	LECTOR/BIBLIOTECARIO/GERENTE
GET /api/v1/prestamos/reportes/libros-mas-prestados	BIBLIOTECARIO/GERENTE
POST /api/v1/reservaciones	LECTOR/BIBLIOTECARIO/GERENTE
GET /api/v1/reservaciones/usuario/{id}	LECTOR/BIBLIOTECARIO/GERENTE
GET /api/v1/multas/usuario/{id}	LECTOR/BIBLIOTECARIO/GERENTE
POST /api/v1/multas/{id}/pago	BIBLIOTECARIO/GERENTE
POST /api/v1/multas/{id}/anulacion	GERENTE/ADMIN

Nótese la asimetría deliberada: `LibroController` sí incluye **ADMIN** en sus 5 endpoints (fix aplicado en esta entrega, ver [Sección 6](#)), pero `PrestamoController` y `ReservacionController` **no** – un usuario exclusivamente **ADMIN** recibe 403 en esos dos controllers (verificado en vivo al construir la colección Postman). Se documenta esta inconsistencia real en vez de asumir que el mismo criterio se aplicó de forma pareja en todo el backend.

2.4 Decisiones arquitectónicas documentadas

El repositorio contiene **10 Architecture Decision Records** (`docs/adr/`), que cubren de forma completa los 6 temas obligatorios del Bloque D de la guía (ver índice en `docs/adr/README.md`) más 2 ADRs adicionales no mapeados a esos temas obligatorios:

ADR	Decisión
ADR-001	Elección de la pila tecnológica principal
ADR-003	Redis para blacklist de tokens JWT revocados
ADR-006	Esquema reproducible (Flyway + <code>schema.sql/seed.sql</code>)
ADR-007	Cookies <code>HttpOnly</code> para JWT – <code>refreshToken</code> migrado, <code>accessToken</code> pendiente
ADR-008	TTL del caché Redis del catálogo, en configuración externa
ADR-009	Licencia MIT del proyecto
ADR-010	JWT stateless + RBAC normalizado sobre un campo de rol simple
ADR-011	PostgreSQL como gestor de base de datos
ADR-012	Docker Compose como estrategia de despliegue
ADR-013	Estrategia híbrida de acceso a datos: CRUD-ORM + stored procedures

Este informe referencia estas decisiones por su identificador cuando resulta relevante, sin reproducir su contenido completo – el detalle de alternativas consideradas, consecuencias y trade-offs vive únicamente en cada ADR, para evitar que este documento y `docs/adr/` diverjan con el tiempo.

3 Arquitectura del sistema (Modelo C4)

El modelo C4 de SGB-SaaS está versionado como código fuente en `docs/arquitectura/workspace.d` (Structurizr DSL), que reemplaza los diagramas estáticos sin fuente versionada de la Entrega 1A (`docs/diagramas/diagrama_c4_capa1.png` y `diagrama_c4_capa2.jpeg`).

3.1 Nivel 1 – Contexto del sistema

Cinco actores interactúan con SGB-SaaS como caja negra: **Lector** (consulta catálogo, reserva, ve su historial), **Bibliotecario** (registra préstamos/devoluciones/multas), **Gerente** (administra personal, catálogos maestros y reportes), **Administrador** (configura parámetros del sistema – rol que *no existía* en el diseño de Entrega 1A, se agregó al normalizar RBAC) y **Visitante anónimo** (solo puede registrarse o iniciar sesión).

3.2 Nivel 2 – Contenedores

Cuatro contenedores, alineados 1:1 con `docker-compose.yml`: Frontend Angular (SPA, sesión en memoria – nunca `localStorage`), Backend Spring Boot (API REST, JWT+RBAC), PostgreSQL (26 tablas) y Redis (blacklist de JWT + caché de catálogo).

Diferencias documentadas respecto al diseño original de Entrega 1A

El propio `workspace.dsl` documenta tres discrepancias reales entre el diseño original y el sistema implementado, en vez de dibujar capacidades que no existen:

- **Gemini 2.0 Flash API** y **Google Books API** aparecían como sistemas externos en el diseño de Entrega 1A. Se **retiraron** del diagrama actual: no

existe ninguna integración con ellos en el código real del backend ni del frontend (verificado por búsqueda en el código fuente).

- **Catálogo público sin autenticación:** el diseño contemplaba que el *Visitante anónimo* pudiera navegar el catálogo sin cuenta. En la implementación real, `LibroController` exige rol `LECTOR` (o superior) incluso para listar libros – el visitante anónimo hoy solo puede registrarse o iniciar sesión. Discrepancia real entre diseño e implementación, documentada explícitamente en vez de oculta.
- **Contenedor Redis y rol Administrador:** ambos ausentes del diagrama de Entrega 1A, agregados en esta entrega como consecuencia directa de la normalización RBAC y de la revocación de tokens vía blacklist.

3.3 Nivel 3 – Componentes

Pendiente para la Entrega Final, por decisión explícita tomada durante el desarrollo de esta entrega – no es un olvido. El propio `workspace.dsl` documenta el motivo: el detalle fino de componentes del backend de Préstamos podía cambiar mientras el módulo seguía en desarrollo activo durante esta entrega, y construir el diagrama de Nivel 3 dos veces (una ahora, otra cuando el módulo se estabilizara) habría sido trabajo perdido. Con el módulo de Préstamos/Reservaciones/Multas ya completo y commitado al cierre de esta entrega, el Nivel 3 puede construirse sobre una base estable en la Entrega Final.

Sobre la exportación a PNG. Los diagramas C4 de Nivel 1 y Nivel 2 existen y están validados sintácticamente (`structurizr/cli validate`, exit code 0), pero **no se incluye una imagen PNG exportada en este informe**: la exportación a PNG está documentada como pendiente de integración al pipeline de CI (Bloque B) directamente en los comentarios de cabecera de `docs/arquitectura/workspace.dsl`, que además deja documentado el procedimiento completo de dos pasos (`Structurizr CLI` → `PlantUML` → `PNG`, o revisión interactiva vía `structurizr/lite`) para cuando se integre. Los diagramas están disponibles en formato Structurizr DSL en `docs/arquitectura/workspace.dsl`, ejecutable/visualizable por cualquier evaluador con Docker.

3.4 Atributos de calidad (ISO/IEC 25010)

`docs/arquitectura/ISO25010.md` prioriza las 8 características de calidad de producto de la norma para el contexto específico de este proyecto (biblioteca universitaria de bajo volumen, no un sistema de alta concurrencia). De las 8, cinco se priorizan como **Alta** (Adecuación funcional, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad, Portabilidad – seis, en realidad, ver el documento fuente para el detalle exacto), con su estrategia concreta de mitigación citada junto al ADR correspondiente cuando existe uno. Un ejemplo representativo: *Fiabilidad* se documenta honestamente como **parcialmente atendida** – ADR-003 deja anotado que, si Redis cae, `JwtAuthFilter` queda sin forma

de verificar revocaciones, y la decisión *fail-open* vs. *fail-closed* sigue **pendiente para producción**, no resuelta.

4 Matriz de trazabilidad

La matriz completa vive en `docs/trazabilidad/matriz.csv` (30 filas, columnas: `id_requisito`, `tipo`, `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso`, `evidencia_empirica`, `estado`) y se valida automáticamente en cada corrida de CI vía `scripts/validate-traceability.sh`, que rechaza el build si algún requisito **Must** carece de historia de usuario o caso de uso, o si algún requisito marcado **verificado** carece de prueba automatizada real. Este informe no reproduce las 30 filas – resume su estado agregado.

4.1 Resumen agregado

Dimensión	Categoría	Cantidad
Tipo	Funcional	16
	No funcional	14
Prioridad MoSCoW	Must	23
	Should	7
Estado	verificado	21
	implementado	7
	pendiente	2
Total		30

4.2 Cruce prioridad × estado

Prioridad	verificado	implementado	pendiente
Must (23)	21	2	0
Should (7)	0	5	2

Ningún requisito Must quedó pendiente: los 23 requisitos de prioridad más alta están al menos **implementado** (21 con evidencia empírica completa + prueba automatizada real, 2 con el código funcionando pero evidencia empírica parcial). Los únicos 2 requisitos genuinamente **pendiente** son de prioridad **Should**: **REQ-NF-012** (transporte TLS, mapeado a OWASP A02) y **REQ-NF-014** (cabeceras de seguridad/CSP en `SecurityConfig` y `nginx.conf`, mapeado a OWASP A05) – ambos reclasificados explícitamente de **Must** a **Should** durante esta entrega, con el criterio de que un entorno de desarrollo sin exposición pública no justifica exigir TLS real hoy (la guía exige un entorno públicamente accesible con TLS recién en la Entrega Final). Este mapeo es consistente con el detalle completo del [Sección 6](#), subsección de Seguridad.

4.3 Trazabilidad de los dos fixes de seguridad de esta entrega

REQ-F-005 y REQ-F-006 (catálogo de libros) pasaron de `implementado` a `verificado` dentro de esta misma entrega, como consecuencia directa del fix de rol `ADMIN` en `LibroController` (ver [Sección 6](#)): antes de ese fix, la evidencia empírica de esos dos requisitos era parcial porque el control de acceso real tenía un hueco no detectado hasta que se verificó en vivo con una cuenta exclusivamente `ADMIN`.

5 Protocolo experimental

5.1 Determinismo y reproducibilidad

Toda evidencia empírica de este proyecto sigue una convención fija, documentada en `docs/mediciones/README.md`:

- **Cabecera obligatoria generada por script, no a mano.** `scripts/mediciones-header.sh` captura, en el momento exacto de la corrida: fecha ISO 8601 UTC real, hash corto del commit, y versiones exactas de Docker, Docker Compose, Java, Maven, PostgreSQL y Redis. Ningún archivo de evidencia completa esta cabecera a mano ni copia la de otro archivo.
- **Convención de nombres.** Evidencia puntual: `YYYY-MM-DD-descripcion-corta.md`. Corridas de k6: `kNN-runM.{json,md}`, donde `NN` identifica el escenario y `M` el número de repetición – nunca se sobrescribe una corrida anterior con el mismo nombre, para poder comparar variabilidad entre corridas.
- **Semilla fija obligatoria** para cualquier dato aleatorio o simulado, documentada en el propio script que la usa (no solo en el archivo de resultados) – convención ya aplicada preventivamente en `k6/opts.js` aunque las corridas de carga aún no se hayan ejecutado.
- **Archivos crudos inmutables.** La edición manual de un archivo de evidencia generado por una herramienta invalida esa evidencia – los archivos de `docs/mediciones/jacoco` (`report.xml`, `report.csv`, `html/`) son la salida exacta de `jacoco-maven-plugin`, sin ningún ajuste posterior.

5.2 Estado de preparación por sub-bloque del Bloque C

Sub-bloque	Configuración lista	Ejecutado
C.1 Rendimiento (k6)	k6/opts.js: perfil de 50 VUs, 30 s de duración sostenida, con <i>ramp-up</i> declarado (no salto directo a 50 VUs). 3 corridas planeadas por escenario.	No
C.2 Seguridad (OWASP)	6 controles auditados en vivo + 2 correcciones con evidencia antes/después.	Sí
C.3 Usabilidad (SUS)	Plantilla de consentimiento informado (docs/etica/consentimientos/plantilla.md), tarea de <i>onboarding</i> de 2 pasos definida, código de participante anónimo (P01, P02, ...). Objetivo: 10 participantes.	No
C.4 Cobertura (JaCoCo)	jacoco-maven-plugin configurado en backend-springboot/pom.xml, exclusiones documentadas.	Sí
C.5 Accesibilidad (Lighthouse)	No hay configuración/script propio en el repositorio todavía.	No

5.3 Por qué C.1, C.3 y C.5 no se ejecutaron en esta entrega

Declarado con la misma honestidad exigida por la guía: el tiempo disponible del equipo para esta Tercera Entrega se concentró en cerrar primero los bloques con mayor peso de evaluación y mayor riesgo si quedaban con evidencia fabricada o inflada – seguridad (C.2) y cobertura (C.4), ambos con hallazgos reales que corregir, no solo medir. Ejecutar k6/SUS/Lighthouse *de forma apresurada* solo para poder marcar la casilla habría producido números sin valor real (una corrida de 50 VUs contra un entorno de desarrollo sin caracterizar previamente, o una prueba SUS con menos de los 10 participantes objetivo, no aportan evidencia confiable) – se prefirió declarar estos tres sub-bloques como **no ejecutados**, con su protocolo completo y reproducible ya preparado, en vez de generar datos de relleno. Ver [Sección 7](#) para el análisis crítico de esta decisión.

6 Resultados por bloque

6.1 Seguridad (OWASP Top 10) – Bloque C.2

Se auditaron 6 controles del OWASP Top 10:2021 en vivo contra el stack Docker real (`docker compose up -d --build`), documentados en docs/mediciones/sec/. Dos de los seis hallazgos iniciales eran gaps reales que **se corrigieron dentro de esta misma entrega**, con evidencia de antes y después contra el mismo comando de verificación.

Control	Hallazgo	Estado
A01 – Control de acceso roto	Acceso cruzado entre usuarios LECTOR (GET /api/v1/prestamos/usuario/{id} de otro usuario) rechazado con 403 real.	PASA
A02 – Fallos criptográficos	Sin TLS activo en el entorno de desarrollo (HTTP plano, docker-compose.yml sin proxy TLS) – exigido recién en Entrega Final. Código sí declara Secure/HttpOnly/SameSite=Strict en la cookie del refreshToken, verificado en la respuesta real.	GAP CONOCIDO
A03 – Inyección	Payloads ' OR '1'='1 y '; DROP TABLE usuarios; - en campos de texto libre de /api/auth/registro: se guardan literalmente como texto, la tabla usuarios permanece intacta (verificado con un login posterior). Todo el acceso a datos usa JPA parametrizado o SPs con parámetros nombrados.	PASA
A05 – Configuración de seguridad	X-Content-Type-Options presentes en backend y frontend. Content-Security-Policy ausente en ambos – omisión de configuración	GAP CONOCIDO

Evidencia cruda seleccionada

Los archivos completos de docs/mediciones/sec/ incluyen cabeceras HTTP íntegras, comandos exactos y análisis de cada corrida; aquí se citan extractos representativos sin editar, para que este informe no dependa únicamente de la tabla-resumen.

A01 – acceso cruzado entre usuarios LECTOR, rechazado:

```
$ curl --include -s -X GET "http://localhost:8080/api/v1/prestamos/usuario/4" \
  -H "Authorization: Bearer $TOKEN_A"

HTTP/1.1 403
Content-Type: application/problem+json

{"detail":"No tiene permisos para realizar esta acción.",
 "instance":"/api/v1/prestamos/usuario/4","status":403,"title":"Forbidden"}
```

A03 – payload ' ; DROP TABLE usuarios; - en campo de texto libre, guardado literalmente, tabla intacta:

```
$ curl --include -s -X POST http://localhost:8080/api/auth/registro \
  -d '{"nombre":"' OR '1'='1",
      "apellido":"' ; DROP TABLE usuarios; --\"," ...}'

HTTP/1.1 201
{"id":5,"nombre":"' OR '1'='1",
 "correo":"usuarioC.owasp@sgb-saas.local","roles":["LECTOR"]}
```

login posterior de un usuario preexistente, para confirmar integridad:
 HTTP_STATUS:200 <-- la tabla "usuarios" sigue intacta

A07 (corregido) – sexto intento consecutivo de login, antes 401, ahora 429; contador real en Redis:

```
--- intento 6 de 6 ---
HTTP/1.1 429
{"detail":"Demasiados intentos fallidos. Intente nuevamente en 898 segundos.",
 "instance":"/api/auth/login","status":429,"title":"Too Many Requests"}
```

```
$ redis-cli GET "login-attempts:usuarioA07fix.owasp@sgb-saas.local:172.18.0.1"
5
$ redis-cli TTL "login-attempts:usuarioA07fix.owasp@sgb-saas.local:172.18.0.1"
884
```

A09 (corregido) – login/logout ahora quedan en el log de aplicación y en bitacora_auditoria:

```
2026-07-30T20:45:02.958Z INFO ... AuthService : Login exitoso:
  sub=13 correo=usuarioReseteo.owasp@sgb-saas.local ip=172.18.0.1
2026-07-30T20:45:32.583Z INFO ... AuthService : Logout:
  correo=usuarioReseteo.owasp@sgb-saas.local
  jti=649f631d-406f-4134-b7f2-4bb43abf09e2 ip=172.18.0.1
```

id	usuario_id	tipo_operacion	detalles	ip_origen
----	------------	----------------	----------	-----------

```

-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
10 |          13 | LOGIN_OK          | Login exitoso para correo: ... | 172.18.0.1

```

Detalle: A02, el gap declarado en vez de simulado

Vale la pena destacar el tratamiento de A02 como ejemplo del criterio de honestidad aplicado en todo el proyecto: el archivo de evidencia correspondiente abre explícitamente diciendo que “*no hay forma honesta de verificar TLS en este entorno porque no existe TLS activo*”, y que “*cualquier evidencia que mostrara un candado o un `https://` funcionando aquí sería fabricada*”. En vez de omitir la sección o fabricar una captura, se documentó qué SÍ es verificable hoy (el código ya declara el atributo `Secure`, listo para activarse el día que exista TLS real sin cambios de código) y qué NO (el propio TLS), dejando el segundo como gap conocido y explícitamente fuera de alcance hasta que exista un entorno públicamente accesible.

Detalle: los dos fixes reales de esta entrega

Más allá de la auditoría OWASP, esta entrega corrigió dos defectos reales adicionales, detectados durante el propio proceso de verificación (no reportados por un tercero):

- **Rol ADMIN faltante en LibroController.** Los 5 endpoints de creación/edición/eliminación de libros no incluían ADMIN en su lista de roles permitidos, pese a que ADMIN es el rol de mayor privilegio en la jerarquía del sistema (ADMIN > GERENTE > BIBLIOTECARIO > LECTOR). Corregido y verificado en vivo con una cuenta exclusivamente ADMIN (docs/mediciones/sec/2026-07-30-owasp-a01-fix-rol-admin-libros.md). Se documentó explícitamente que PrestamoController y ReservacionController tienen el mismo hueco y quedan **sin corregir**, fuera del alcance de ese fix puntual.
- **POST /api/auth/refresh respondía 500 en vez de 401** cuando el refreshToken de la cookie era inválido/expirado: una RuntimeException genérica sin @ExceptionHandler dedicado caía en el handler de último recurso. Corregido con una excepción específica (RefreshTokenInvalidoException) y su handler en GlobalExceptionHandler, devolviendo 401 vía ProblemDetail (RFC 7807).

6.2 Cobertura de código (JaCoCo) – Bloque C.4

Reporte real generado por `jacoco-maven-plugin` (versión 0.8.12) sobre el dominio/servicios del backend, excluyendo `dto/` (records planos sin lógica), `entity/` (entidades JPA con `@Data`, sin métodos de negocio propios) y `config/` (definición de beans de framework) – criterio documentado también como comentario en `backend-springboot/pom.xml`. Artefactos completos versionados en `docs/mediciones/jacoco/` (`report.xml`, `report.csv`, `html/`).

Totales agregados

Métrica	Cubierto	Total	%
Líneas	380	501	75,85 %
Ramas	50	102	49,02 %
Complejidad	111	190	58,42 %

El objetivo de esta entrega (**≥ 60 % de líneas**) se supera con margen, y el resultado ya alcanza el umbral fijado para la Entrega Final (≥ 70 %). Verificado con `./mvnw clean verify`: BUILD SUCCESS, 79/79 tests (0 fallos, 0 errores).

Desglose por paquete

Paquete	Líneas	Ramas
security	86/90 (95,6 %)	15/22 (68,2 %)
scheduling	7/7 (100,0 %)	–
controller	46/66 (69,7 %)	3/4 (75,0 %)
service	224/304 (73,7 %)	30/62 (48,4 %)
exception	17/34 (50,0 %)	2/14 (14,3 %)

Historia: de 53,69 % a 75,85 %

La primera corrida real de JaCoCo en esta entrega arrojó 53,69 % de líneas – por debajo del objetivo. En vez de agregar tests triviales para inflar el número (instrucción explícita seguida durante todo el proceso), se identificaron 5 clases de autenticación/autorización con cobertura casi nula *dentro del proceso de test* porque los tests existentes las mockeaban en vez de ejercitarlas: `JwtService`, `JwtAuthFilter`, `UserDetailsServiceImpl`, `AuthController` y `GlobalExceptionHandler`. Se agregaron 4 clases de test nuevas que ejercitan la implementación *real* de cada una (`JwtServiceTest` unitario puro sin contexto Spring, `JwtAuthFilterTest` con el filtro real y solo Redis/BD mockeados, `UserDetailsServiceImplTest`, y `AuthControllerTest` vía `MockMvc + springSecurity()` real, mismo patrón que `LibroControllerSecurityTest`):

Clase	Antes	Después
<code>JwtService</code>	4,8 %	95,2 %
<code>JwtAuthFilter</code>	18,2 %	100,0 %
<code>UserDetailsServiceImpl</code>	7,1 %	100,0 %
<code>AuthController</code>	0,0 %	100,0 %
<code>GlobalExceptionHandler</code>	8,8 %	50,0 %

`GlobalExceptionHandler` se dejó deliberadamente en 50 %: las líneas restantes corresponden a `handleNotFound` (`EntityNotFoundException`, no aplica a ningún flujo de `AuthController`) y al *catch-all* de último recurso (`handleGenerica`) – forzar esas líneas habría exigido tests artificiales fuera del alcance de auth/seguridad, exactamente el tipo de relleno que se pidió evitar.

Detalle completo por clase (las 20 clases analizadas)

Tabla completa de docs/mediciones/jacoco/report.csv, ordenada de menor a mayor cobertura de líneas – ninguna clase se omite:

Paquete	Clase	% líneas
controller	PrestamoController	25,0 %
controller	MultaController	42,9 %
controller	ReservacionController	42,9 %
controller	TestController	50,0 %
exception	GlobalExceptionHandler	50,0 %
service	PrestamoService	66,1 %
service	AuthService	67,5 %
service	MultaService	72,7 %
service	LibroService	75,7 %
controller	LibroController	80,0 %
security	LoginRateLimiter	83,3 %
service	ReservacionService	90,2 %
security	JwtService	95,2 %
controller	AuthController	100,0 %
scheduling	ReservacionScheduler	100,0 %
security	JwtAuthFilter	100,0 %
security	UserDetailsServiceImpl	100,0 %
service	CorreoYaRegistradoException	100,0 %
service	LoginRateLimitExcedidoException	100,0 %
service	RefreshTokenInvalidoException	100,0 %

6.3 Rendimiento – Bloque C.1 (k6)

NO EJECUTADO

No ejecutado en esta entrega. La configuración base ya está lista en k6/opts.js: perfil de 50 VUs, 30 s de duración sostenida, con etapas de *ramp-up/ramp-down* declaradas en vez de saltar directo a la carga máxima, y 3 corridas planeadas por escenario para poder comparar variabilidad (docs/mediciones/perf/kNN-runM.json). El script de prueba en sí (las funciones que llaman a los endpoints, los *checks* y los *thresholds* de negocio) queda pendiente de escribir antes de la primera corrida real. Ningún número de latencia/throughput se reporta en este informe para no fabricar evidencia que no existe.

6.4 Usabilidad – Bloque C.3 (SUS)

NO EJECUTADO

No ejecutado en esta entrega. El protocolo completo está preparado: plantilla de consentimiento informado en docs/etica/consentimientos/plantilla.md (código

de participante anónimo tipo P01, tarea de *onboarding* de dos pasos – iniciar sesión con cuenta de prueba, registrarse como usuario nuevo – sin ayuda del equipo salvo bloqueo total), objetivo de 10 participantes, y la política de anonimización ya declarada en docs/etica/ETHICS.md (formularios firmados nunca se committean al repositorio; solo la plantilla en blanco). Ningún puntaje SUS se reporta en este informe.

6.5 Accesibilidad – Bloque C.5 (Lighthouse)

NO EJECUTADO

No ejecutado en esta entrega. A diferencia de C.1 y C.3, este sub-bloque todavía no tiene ni siquiera configuración base preparada en el repositorio – es el sub-bloque con menor avance de los tres pendientes. Queda como trabajo a iniciar desde cero para la Entrega Final: definir el conjunto de páginas del frontend Angular a auditar y los umbrales mínimos aceptables por categoría (Accesibilidad, Rendimiento, Buenas prácticas, SEO).

7 Amenazas a la validez

Análisis crítico real de las limitaciones de esta entrega, no una sección de relleno:

1. **Tres de cinco sub-bloques de evidencia empírica del Bloque C sin ejecutar.** Rendimiento (C.1), usabilidad (C.3) y accesibilidad (C.5) quedaron como protocolo/configuración lista pero sin corrida real, por restricción de tiempo del equipo dentro de esta entrega (ver Sección 5). Esto significa que las afirmaciones de *eficiencia de desempeño* en docs/arquitectura/ISO25010.md se apoyan únicamente en una medición puntual de caché (170ms *miss* vs. 30ms *hit*), no en una prueba de carga formal – no hay evidencia todavía de cómo se comporta el sistema bajo 50 VUs concurrentes.
2. **Escala del equipo limita la escala de las pruebas de usabilidad.** El equipo tiene 3 integrantes, todos con roles activos de desarrollo durante esta entrega – reclutar y coordinar 10 participantes externos para SUS (objetivo declarado en el protocolo) compite directamente por el mismo tiempo limitado que exige cerrar los demás bloques. Esta es una limitación estructural del proyecto, no solo de esta entrega puntual.
3. **Auditoría de seguridad manual, no exhaustiva.** Las 6 verificaciones OWASP se hicieron con curl dirigido a escenarios específicos elegidos por el equipo, no con una herramienta de *fuzzing*/escaneo automatizado (ej. OWASP ZAP) que explore superficie de ataque no anticipada. Además, varios hallazgos se verificaron solo en un endpoint representativo y se documentó explícitamente que el mismo patrón se asume (no se re-verificó) en endpoints hermanos – ej. A01 se probó solo contra PrestamoController, asumiendo que MultaController/ReservacionController comparten el mecanismo (validarAccesoUsuario), sin repetir la prueba en cada uno.

4. **Gap de validación en ReservacionService (hallazgo nuevo, no corregido).** Detectado al construir la colección Postman de esta entrega: `ReservacionService.crear` no valida que `libroId` exista antes de guardar (a diferencia de `PrestamoController`, que sí pasa por un procedimiento almacenado con validación LB404). Una reserva con un libro inexistente no responde 404: la restricción de llave foránea de PostgreSQL revienta como excepción no mapeada, y `GlobalExceptionHandler` la convierte en **500 Internal Server Error**. Verificado en vivo y documentado en `docs/postman/coleccion.json`, no corregido – fuera del alcance del prompt que lo detectó.
5. **CONTRIBUTORS.md desactualizado respecto al estado real del repositorio.** El propio archivo declara, en su nota de cabecera, que el backend de Préstamos/Reservas y los ajustes de frontend correspondientes aún no estaban commiteados al momento de escribirse – al cierre de esta entrega, ese trabajo sí está commiteado, pero el archivo no se ha vuelto a revisar. Se documenta aquí en vez de inventar una actualización que no se verificó línea por línea contra `git log` en el momento de escribir este informe.
6. **TLS y CSP diferidos, no resueltos.** OWASP A02 (TLS en tránsito) y parte de A05 (*Content-Security-Policy*) quedan como gap conocido explícito – correcto para un entorno de desarrollo sin exposición pública, pero significa que la postura de seguridad completa del sistema no puede evaluarse hasta que exista el entorno públicamente accesible que exige la Entrega Final.

8 Contribuciones y declaración ética

8.1 Contribuciones (taxonomía CRediT)

El detalle completo de roles por integrante, basado en trabajo realmente commiteado (no en el rol nominal de equipo), vive en `CONTRIBUTORS.md` siguiendo la taxonomía CRediT (14 roles estándar, <https://credit.niso.org/>). Resumen:

- **Marlon Loor Medranda** (Tech Lead / DevOps / Seguridad): *Software* (JWT+RBAC, cookies `HttpOnly`, blacklist de tokens, caché con TTL externo, `GlobalExceptionHandler`), *Project administration*, *Supervision* (revisión de hallazgos de QA sobre trabajo de otros integrantes) y *Validation* (verificación en vivo de cada cambio de seguridad/caché contra el stack Docker real).
- **Irvin Cajas Ibarra** (Backend): *Software* – CRUD de libros y el módulo de Préstamos/Reservas/Multas.
- **Moises Panama Murillo** (Frontend): *Software* – módulos Angular de autenticación (login/registro, guards, interceptor JWT) y CRUD de libros en el frontend.

8.2 Declaración ética

`docs/etica/ETHICS.md` cubre las cuatro declaraciones exigidas por el Bloque F de la guía:

1. **Procedencia de datos.** Los 5 libros de ejemplo de `db/seed.sql` **no son ficticios**: son libros reales publicados, con ISBN real (ej. *Clean Code*, ISBN 9780132350884). Los metadatos bibliográficos (ISBN, título, autor, editorial) son hechos públicos de catalogación, transcritos manualmente por el equipo – sin *scraping* ni consumo de API de terceros con licencia. El resumen de una línea por libro es redacción original del equipo. El contenido íntegro de ninguna obra está incluido en el repositorio.
2. **Tratamiento de datos personales.** Contraseñas hashadas con BCrypt costo 12, nunca en texto plano. Auditoría explícita del código fuente confirmó que `password_hash` no se expone por ningún endpoint ni DTO (`@JsonIgnore` sobre el campo, y el único DTO de registro no incluye ese campo en su definición).
3. **Consentimiento informado para SUS.** Mecanismo declarado *antes* de la primera prueba, no retroactivamente – ver [Sección 5](#).
4. **Ausencia de datos identificables en el repositorio público.** `docs/mediciones/` solo contiene evidencia técnica (códigos de estado HTTP, *timings*, TTLs de Redis); los consentimientos firmados de participantes reales de SUS nunca se committean, solo la plantilla en blanco.

9 Referencias

1. ISO/IEC/IEEE 29148:2018 – *Systems and software engineering – Life cycle processes – Requirements engineering*.
2. ISO/IEC 25010:2011 – *Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*.
3. RFC 7519 – *JSON Web Token (JWT)*, M. Jones, J. Bradley, N. Sakimura, IETF, mayo 2015.
4. RFC 7807 – *Problem Details for HTTP APIs*, M. Nottingham, E. Wilde, IETF, marzo 2016.
5. OWASP Top 10:2021 – *The Ten Most Critical Web Application Security Risks*, OWASP Foundation. <https://owasp.org/Top10/>
6. Semantic Versioning 2.0.0. <https://semver.org/>
7. Keep a Changelog 1.1.0. <https://keepachangelog.com/>
8. CReDiT – Contributor Roles Taxonomy. NISO/CASRAI. <https://credit.niso.org/>
9. Brooke, J. (1996). *SUS: A “quick and dirty” usability scale*. En P.W. Jordan et al. (Eds.), *Usability Evaluation in Industry*. Taylor & Francis.

10. Brown, S. – *The C4 model for visualising software architecture*. <https://c4model.com/>
11. Google – *Lighthouse*, herramienta de auditoría automatizada de calidad web. <https://developer.chrome.com/docs/lighthouse/>
12. Grafana Labs – *k6*, herramienta de pruebas de carga. <https://k6.io/>

Este informe se generó a partir de evidencia real versionada en el repositorio del proyecto (<https://github.com/mloorm14/sgb-saas>, tag v0.9.0-rc, commit c6b372c). Ningún dato de rendimiento, usabilidad o accesibilidad reportado en este documento fue inventado o extrapolado – donde la evidencia no existe (Bloques C.1, C.3, C.5), se declaró explícitamente como no ejecutada.